

Data Science Questions

“At end of each Chapter”

Chapter 1 “Intro to Data Science”

1. What are the foundations of data science?

◆ The main components of Data Science :

- 1) **Statistics**: Analyzing large numerical data and find valuable insights
- 2) **Domain Expertise**: Applying data science to specific industries
- 3) **Data Engineering**: Managing, storing, and transforming data
- 4) **Visualization** : Presenting data in a human-readable format
- 5) **Advanced Computing** : Using AI, machine learning, and deep learning for prediction

2. Discuss the areas where data science is applicable?

◆ Data Science is applied across industries for:

- **Healthcare** : Analyzing patient data for treatment recommendations
- **Finance** : Fraud detection and risk analysis
- **Marketing** : Customer behavior prediction
- **IOT** : Managing sensor-generated data
- **Elections** : Optimizing campaign strategies

3. Discuss the various challenges due to growth in data (characteristics of big data)

◆ Five V's of Big Data (Challenges) :

- 1) **Volume** : refers to the enormous size of data
- 2) **Velocity** : refers to the high speed of accumulation of data
- 3) **Variety** : refers to nature of data. Data is available in varied formats and heterogenous sources (structured/un-structured/semi-structured)
- 4) **Veracity** : refers to inconsistencies & uncertainty in data (data quality)
- 5) **Value** : refers to data that have valuable insights that can be extracted

4. Explain briefly the data Science Process.

- 1) **Business Understanding**: Define research goals and project charter
- 2) **Data Retrieval**: Collect from internal/external sources
- 3) **Data Preparation**:
 - a. **Cleansing**: Handle missing values, outliers
 - b. **Integration**: Combine datasets
 - c. **Transformation**: Standardize formats
- 4) **EDA**: Explore data distributions, relationships
- 5) **Modeling**:
 - a. Select algorithms
 - b. Train, test, and compare models
- 6) **Presentation**: Deliver insights via reports/dashboards
- 7) **Deployment**: Test in pre-production, then deploy for real-time use.

5. Define Messy data, and give examples

- ◆ **Missy Data** : is data that is not in usable form or it is impossible to obtain clearly information from it

- ◆ It needs lot of preparing/preprocessing to be usable

Ex : Duplicate records with inconsistent spellings - column mixing two different data – missing data

Q: Give examples of messy data and how to manipulate it ? “Midterm”

Example	How to Manipulate it
Missing values (NaN or blanks)	Fill or drop missing data “df.dropna()”
Duplicate rows	Remove duplicates “df.drop_duplicates()”
Mixed data types in a column	Convert to a single data type
Combined data in one column	Split into multiple columns
Irregular date formats	Convert to datetime

Chapter 2 “Data Acquisition and Processing”

6. What are the various types of data?

- a) **Structured Data:** The data which is available in some stand format, Organized in rows/columns (relational databases, Excel) – Ex: [Names, addresses, geolocations]
- b) **Unstructured Data:** unorganized data with no particular format (text, images, videos). Require processing to be understandable
- c) **Semi-Structured data:** a semi-organized data, don't conform to the formal structure of data – Ex : log files

7. Explain how to handle missing Data ?

- **Imputation:** Guess missing values (fill gender based on other entries)
- **NA/NaN:** Use fillna() in Pandas for non-existent values
- **Deletion:** Use dropna() if values cannot be figured

8. What's Noisy Data? And list techniques to handle it .

- **Noisy Data:** Random variance in the data
- **Techniques :**
 - I. **Binning method (discretization) :**
 - a. Sort data.
 - b. Partition into equal-frequency bins (no. elements / bin depth)
 - c. Replace each values with bin mean/median/boundary
 - II. **Regression:** Fit a mathematical equation (linear/multiple) to smooth noise
 - III. **Clustering:** Group similar data to detect outliers

9. Explain the concept of data preprocessing. What are the techniques used for preprocessing of data?

- **Data preprocessing** transforms raw/inconsistent data into a usable format by addressing issues like missing values, noise, and redundancies.
- **Techniques:**
 - a. **Data Cleaning :** Remove duplicates/irrelevant data – fix structural errors – handle missing data – smooth noise
 - b. **Data Integration :** Combines data from multiple sources (ETL, data virtualization, warehousing)
 - c. **Data Transformation :** Standardizes data into main form
 - d. **Data Reduction :** Dimensionality reduction -- Numerosity reduction -- Data compression

10. Write a short note on Data mining.

- Involves in extracting information from a given data set to identify trends, patterns, and useful data.

11. What are the techniques used for data mining?

- **Association:** Uses if-then rules ("If X, then Y").
- **Clustering:** Groups data by similarity (customer segments).
- **Classification:** Assigns labels
- **Prediction:** Uses regression to forecast outcomes.
- **Sequential Patterns:** Analyzes ordered sequences (e.g., purchase history).

12. What is the importance of data Interpretation?

- assign some meaning to the data and arrive at a relevant conclusion using some predefined processes

Chapter 3 “Data Representation”

13. Discuss the various representation techniques of different data types.

- 1) **Structured Data:** Tabular formats (RDBMS, Excel) with fixed fields and defined data types
- 2) **Unstructured Data:** No predefined model (emails, multimedia).
- 3) **Natural Language Data:** Processed via NLP techniques (entity recognition, sentiment analysis).
- 4) **Machine-Generated Data:** Autonomously produced (server logs, IoT sensor outputs).
- 5) **Streaming Data:** Real-time processing (Twitter trends, stock markets).
- 6) **Acoustic/Video/Image Data:**
 - **Images:** Vector (scalable lines/curves) or bitmap (pixel arrays).
 - **Audio:** Lossy (MPEG) or lossless (WMA) compression.
- 7) **Sensor Data:** IoT device inputs (environmental metrics).
- 8) **Graph/Network Data:** Nodes/edges represent relationships (social networks).

14. What are the issues that are faced in handling large data?

- **Lack of Understanding:** Companies fail to leverage data effectively.
- **High Costs:** Infrastructure and expertise expenses.
- **Too Many Choices:** Overwhelming tool options ("paradox of choice")
- **Complex Systems:** Fragmented data management.
- **Security Gaps:** Data breaches and bias risks.
- **Low-Quality Data:** Inconsistent formatting, missing/duplicate entries.
- **Compliance Hurdles:** GDPR and regulatory risks.

15. Explain briefly the techniques to handle large data

- 1) **Choosing the right Algorithms:**
 - ❖ **Online Algorithms:** Incremental processing ("use and forget").
 - ❖ **Block Algorithms:** Parallel computation (MapReduce).
 - ❖ **Learning Modes:** Full-batch, mini-batch, or online
- 2) **Choosing the right Data Structure :**
 - ❖ **Sparse Data:** Minimal storage (binary word vectors).
 - ❖ **Trees:** Hierarchical retrieval (root → leaf nodes)
 - ❖ **Hash Tables:** Key-value pairs (Python dictionaries)
- 3) **Choosing the right Tool :**
 - ❖ **Cython:** Speed optimization via type declarations.
 - ❖ **Numexpr:** Faster numerical operations.
 - ❖ **Numba:** Just-in-time compilation.
 - ❖ **Bcolz:** Compressed arrays.

16. What are the popular frameworks for big data storage?

1. **Hadoop:**
 - **Modules:** Common, HDFS, YARN, MapReduce.
 - **Splits data across clusters for parallel processing**
2. **Apache Spark:**
 - **Hybrid batch/stream processing.**
 - **Uses Resilient Distributed Datasets (RDDs).**
 - **Components:** SQL, Streaming, MLlib, GraphX
3. **Apache Storm:** Real-time stream processing via Directed Acyclic Graphs (DAGs)
4. **Samza:** Asynchronous streams with Kafka/YARN
5. **Flink:** Hybrid batch/stream with fault tolerance

Chapter 4 “DATA ACQUISITION AND PROCESSING 2”

17. Write a short note on Data Science Ethics.

- **Data science ethics** involves ensuring responsible handling of data to prevent harm. Professionals must report unethical practices like data theft, improper storage, or misuse.
- **Key measures:**
 - **Cybersecurity:** Regular audits, secure computing, and policies for protection
 - **Corporate Culture:** Prioritize ethics over short-term profits
 - **Public Trust:** Users prefer companies that treat data responsibly

18. What are the privacy aspects of data?

Privacy aspects classify data into:

1. **Classified/National Security Data:** Highly regulated (military, infrastructure).
2. **Proprietary Data:** Intellectual property
3. **Personal Data (PII) : Examples:** Social security numbers, medical records, location
4. **High-Risk Groups:** Activists/journalists need secure data handling to prevent harm.

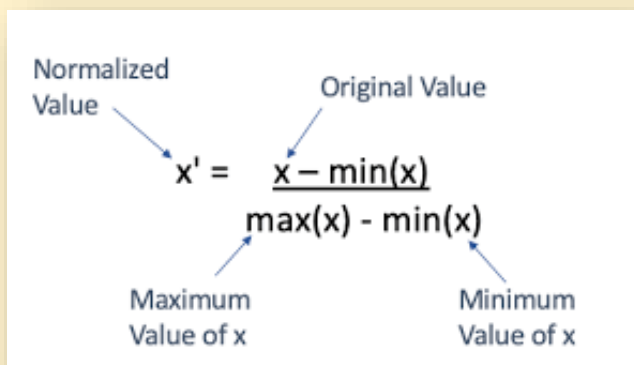
19. What are the five C's of data Science?

- **The Five C's guide ethical data products:**
 1. **Consent:** Clear agreements on data usage
 2. **Clarity:** Transparent explanations
 3. **Consistency & Trust:** Predictable behavior
 4. **Control & Transparency:** User autonomy
 5. **Consequences:** Assess harms

20. Define Data Normalization, and give example of its techniques ?

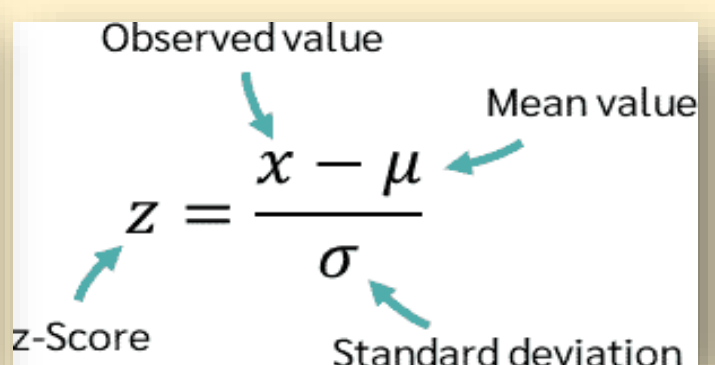
- **Data normalization** is the practice of organizing data entries to ensure they appear similar across all fields and records, making information easier to find, group and analyze.
- **Techniques:**

Min-Max Normalization



The diagram shows the formula for Min-Max Normalization: $x' = \frac{x - \min(x)}{\max(x) - \min(x)}$. Arrows point from labels to parts of the formula: 'Normalized Value' points to x' , 'Original Value' points to x , 'Maximum Value of x' points to $\max(x)$, and 'Minimum Value of x' points to $\min(x)$.

Z-Score Normalization



The diagram shows the formula for Z-Score Normalization: $Z = \frac{x - \mu}{\sigma}$. Arrows point from labels to parts of the formula: 'Observed value' points to x , 'Mean value' points to μ , 'z-Score' points to Z , and 'Standard deviation' points to σ .

Chapter 5

“DATA WRANGLING COMBINING AND MERGING DATA SETS”

21. What is data wrangling?

- **Data wrangling** is the process of transforming raw data into a structured and usable format for analysis. It involves collecting, cleaning, enriching, and validating data to ensure high quality and readiness for further processing.
- **Also known as** : Data Munging or Data Pre-Processing

22. List Steps for Data Wrangling ? “what’s data cleaning {one of the steps}“

- The data wrangling process consists of six core activities:
 1. **Discovery** : Understanding the dataset to identify trends and patterns
 2. **Structuring** : Reformatting data (splitting columns, pivoting rows)
 3. **Cleaning** : Handling missing values, duplicates, and mismatched data
 4. **Enriching** : Combining multiple datasets (joins, aggregations)
 5. **Validating** : Ensuring data accuracy post-transformation
 6. **Publishing** : Making the processed data available for analytics

23. List Tools for Data Wrangling ?

- **Python & R** : Primary programming languages for data manipulation
- **Tabula** : Extracts structured data tables from PDFs into CSV/Excel
- **Data Wrangler** : Cleans and transforms raw data into structured tables
- **OpenRefine** : Java-based tool for data cleaning and format conversion
- **CSVKit** : Command-line utility for CSV manipulation

24. What is merging dataset?

- **Merging is** the process of bringing two or more datasets together into a single dataset and aligning the rows based on common attributes or columns. It uses **merge()** in pandas to combine datasets based on keys or common columns

25. What is reshaping dataset?

- **Reshaping is** the process of changing the shape or structure of a dataset, such as converting wide data tables into long ones using functions like **melt()**, **stack()**, **unstack()**, and **pivot()**

26. Explain concat() function with example

- The **concat()** function in pandas is used to combine datasets across rows or columns.

```
'''
pd.concat(objs, axis, join, join_axes, ignore_index, keys)

# objs : list of DataFrame or Series objects to concatenate
# axis : {0, 1, 'index', 'columns'}, default 0
# join : {'inner', 'outer'}, default 'outer'
# join_axes : list of Index objects, default None
# ignore_index : bool, default False
# keys : sequence, default None
'''
```

```
# example
import pandas as pd
df1 = pd.DataFrame({
    "Name": ["Rahul", "Shreya", "Pankaj", "Monika", "Kalpesh"],
    "Age": [20, 21, 22, 23, 24] })
df2 = pd.DataFrame({
    "Name": ["Mayank", "Jalpa", "Sanjana", "Vimal", "Raj"],
    "Age": [25, 26, 27, 28, 29] })
df3 = pd.concat([df1, df2]) # Default concatenation along rows
df3 = pd.concat([df1, df2], axis=1) # Concatenation along columns
df3 = pd.concat([df1, df2], keys=['x','y'], ignore_index=True)
print(df3)
```


27. Explain merge operation with syntax and example

- The **merging/joining** is the process of bringing two or more datasets together into single dataset and aligning the rows from each dataset based on the common attributes or columns

```
...
pd.merge(left, right, how, on, left_on, right_on, left_index, right_index, sort)

left : is the left DataFrame to merge
right : is the right DataFrame to merge
how : is the type of merge to be performed, default is 'inner'.
on : is the column or index level names to join on.
     These must be found in both DataFrames.
left_on : is the name of column from left data frame to use as keys.
right_on : is the name of column from right data frame to use as keys.
...

# example
import pandas as pd
df1 = pd.DataFrame({
    "Name": ["Rahul", "Shreya", "Pankaj", "Monika", "Kalpesh"],
    "Age": [20, 21, 22, 23, 24] })
df2 = pd.DataFrame({
    "Name": ["Mayank", "Jalpa", "Sanjana", "Vimal", "Raj"],
    "Age": [25, 26, 27, 28, 29] })

inner_join = pd.merge(df1, df2, on="Name", how="inner")
outer_join= pd.merge(df1, df2, on="Name", how="outer")
```

28. Explain reshaping dataset different functions

- **melt()**: Converts wide format to long format.
- **stack()**: Stacks columns as rows (increases index level).
- **unstack()**: Reverses the effect of **stack()**.
- **pivot()**: Reshapes data based on specific column

29. Explain string function with example

- String functions help manipulate and analyze strings.
- Example :
 - str = “Data Science”
 - Print(str.**upper**()) # Converts all characters to uppercase
 - Print(str.**lower**()) # Converts all characters to lowercase
 - Print(str.**replace**(“Science”,”Analysis ”)) # Replace 'Science' with 'Analytics'
 - Print(str.**count**(“a”)) # Counts occurrences of 'a' in the string

30. Explain regular expression with example

- Regular expressions (RegEx) are used to search, match, or manipulate text patterns
- Example :
 - Import re
 - string = “Data Science”
 - print(**re.findall**("th",string)) # return list contains all match values.
 - print(**re.search**("wi",string)) # return first match value occured
 - print(**re.split**("\\s",string)) # split string at each white space “\\s”
 - print(**re.sub**("\\s", "-",string)) # replace each white space with “-”

“Codes Questions”

Question	Code
Create and display a data frame	<pre>import pandas as pd df = pd.DataFrame({ "Name": ["Jayesh", "Priya"], "City": ["Rajkot", "Delhi"] }) print(df)</pre>

Create a data frame and find null values and remove it	<pre>import pandas as pd df = pd.DataFrame({"Age": [21, None, 24]}) print(df.isna().sum()) df = df.dropna() print(df)</pre>
Create a data frame and inert new column in data frame	<pre>import pandas as pd df = pd.DataFrame({"Name": ["Jayesh", "Priya"]}) df.insert(1, "Gender", ["Male", "Female"]) print(df)</pre>
Create a data frame and convert categorical data into numerical values	<pre>import pandas as pd df = pd.DataFrame({ "Name": ["John", "Alice"], "Gender": ["Male", "Female"]}) df["Gender"] = df["Gender"].replace({"Male": 0, "Female": 1}) print(df)</pre>
Create a data frame and split the column	<pre>import pandas as pd df = pd.DataFrame({ "Name": ["Jayesh Patel", "Priya Shah", "Vijay Sharma"] }) df[['First Name', 'Last Name']] = df.Name.str.split(expand=True) print(df)</pre>
Create data frames and combine using concat function	<pre>import pandas as pd df1 = pd.DataFrame({ "Name": ["Rahul", "Shreya"], "Age": [20, 19] }) df2 = pd.DataFrame({ "Name": ["Mayank", "Jalpa"], "Age": [22, 21] }) df3 = pd.concat([df1, df2]) print(df3)</pre>
Create data frames and merge with different arguments	<pre>import pandas as pd left = pd.DataFrame({ "ID": [1, 2, 3, 4, 5], "Name": ["Alice", "Bob", "Charlie", "David", "Eve"], "Course": ["B.E.", "B.Tech.", "MCA", "MBA", "B.Tech."] }) right = pd.DataFrame({ "ID": [1, 2, 3, 4, 5], "Name": ["Alice", "Bob", "Charlie", "David", "Eve"], "Course": ["B.Tech.", "MCA", "MBA", "B.E.", "B.Tech."], }) # Merge with left join print(pd.merge(left, right, on="Course", how="left")) # Merge with right join print(pd.merge(left, right, on="Course", how="right")) # Merge with inner join print(pd.merge(left, right, on="Course", how="inner")) # Merge with outer join print(pd.merge(left, right, on="Course", how="outer"))</pre>
Create data frame and reshape using melt function	<pre>import pandas as pd df = pd.DataFrame({ "ID": [1, 2, 3, 4, 5], "Name": ["Alice", "Bob", "Charlie", "David", "Eve"], "Course": ["B.Tech.", "MCA", "MBA", "B.E.", "B.Tech."], }) print(df.melt())</pre>
Perform string manipulation operations on string	<pre>str = "Data Science" print(str.upper()) # 'DATA SCIENCE' print(str.lower()) # 'data science' print(str.replace("Science", "Analytics")) # 'Data Analytics' print(str.split(" ")) # ['Data', 'Science'] print(str.count("a")) # 2</pre>
Perform regular expression functions on string	<pre>import re string = "Working with Data Science using Python" # Find all "th" print(re.findall("th", string)) # ['th', 'th'] # Search for "wi" print(re.search("wi", string)) # <re.Match object...> # Split by space print(re.split("\s", string)) # ['Working', 'with', '...'] # Replace space with dash print(re.sub("\s", "-", string)) # 'Working-with-Data-Science-using-Python'</pre>

Chapter 6 : AGGREGATION AND GROUP OPERATIONS

31. What is data aggregation?

- **Data aggregation** is the process of gathering raw data and expressing it in summarized form for statistical analysis. It helps to process large datasets efficiently by summarizing data with statistical functions like sum, min, max, average, and count.

32. What is data splitting?

- **Data splitting** is the process of dividing a dataset into different groups based on specific criteria. It is the first step in the groupby operation, where data is split into groups before applying functions to each group independently.

33. What is transformation?

- **Transformation** is a group-specific computation that returns a like-indexed result. It is applied using the **transform()** function and returns data that has the same size as the group being transformed.

34. What is pivot table?

- **A pivot table** is a statistical table that summarizes a large dataset by aggregating numeric data. It provides insights by grouping data based on specified columns and applying functions like sum, min, max, mean...

35. What is cross tabulation?

- **Cross-tabulation** is used to compute the frequency distribution of variables. It's implemented using the **pd.crosstab()** function, which builds a table showing the relationships between two or more categorical variables.

36. Explain data aggregation with different functions.

- Data aggregation involves summarizing data using functions like:
 - **Sum:** Adds all values (df.sum()).
 - **Min:** Finds the lowest value (df.min()).
 - **Max:** Finds the highest value (df.max()).
 - **Average:** Computes the mean (df.mean()).
 - **Count:** Counts entries (df.count()).
 - **Applied using** → **df.agg(["sum"])**

37. Explain groupby function with example.

- The groupby() function splits data into groups, applies functions (e.g., sum, mean), and combines results :
 - **df = pd.DataFrame({"Product": ["Mango", "Corn"], "Price": [350, 80]})**
 - **print(df.groupby("Product").sum())**

38. Explain transform function with syntax and example.

- The transform() function applies operations to groups and returns results of the same size.
 - **df.transform(func = lambda x : x * 5) # Multiply each element by 5**

39. Explain pivot table with example.

- A pivot table summarizes data by grouping and aggregating.
- `df.pivot_table(index="Product", values="Price", aggfunc="sum")`

40. Explain cross tabulation with example.

- Cross-tabulation shows frequency distributions of categorical variables.
- `pd.crosstab(df.Subject, df.Result, margins=True)`

41. Explain date and time data type with different format code.

- Python uses the datetime module with classes like date, time, and datetime. It supports various formats using:
 - %Y: Year
 - %m: Month
 - %d: Day
 - %H:%M:%S: Time

“Codes Questions”

Question	Code
Create a data frame and perform aggregation functions.	<pre>import pandas as pd df = pd.DataFrame({ 'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'], 'Age': [24, 27, 22, 32, 29], 'Score': [85, 90, 88, 76, 95] }) print("Mean Age:", df['Age'].mean()) print("Max Score:", df['Score'].max())</pre>
Create data frames and perform groupby function with different arguments.	<pre>import pandas as pd df = pd.DataFrame({ "Product": ["Mango", "Corn", "Orange", "Cabbage"], "Category": ["Fruit", "Vegetable", "Fruit", "Vegetable"], "Qty": [12, 5, 10, 2], "Price": [350, 80, 320, 50] }) # Group by Product and get mean Price print(df.groupby("Product")["Price"].mean()) # Group by Category and get sum of Price print(df.groupby("Category")["Price"].sum())</pre>
Create data frames and perform transform function	<pre>import pandas as pd df = pd.DataFrame({ "A": [8, 7], "B": [5, 22]}) transformed = df.transform(lambda x: x*5) print(transformed)</pre>
Create data frame and perform pivot table with different arguments	<pre>import pandas as pd # Create DataFrame df = pd.DataFrame({ "Product": ["Mango", "Corn", "Orange", "Cabbage", "Mango", "Corn", "Watermelon", "Apple", "Pumkin", "Mango"], "Category": ["Fruit", "Vegetable", "Fruit", "Vegetable", "Fruit", "Vegetable", "Fruit", "Fruit", "Vegetable", "Fruit"], "Qty": [12, 5, 10, 2, 10, 3, 5, 8, 2, 10], "Price": [350, 80, 320, 50, 200, 50, 280, 380, 60, 400] }) # Pivot table: Total price per product print(df.pivot_table(index="Product", values="Price", aggfunc="sum")) # Pivot table: Total price per category print(df.pivot_table(index="Category", values="Price", aggfunc="sum"))</pre>

Create data frame and prepare two-way and three-way cross table

```
import pandas as pd

# Create DataFrame
df = pd.DataFrame({
    "Product": ["Mango", "Corn", "Orange", "Cabbage", "Mango",
                "Corn", "Watermelon", "Apple", "Pumkin", "Mango"],
    "Category": ["Fruit", "Vegetable", "Fruit", "Vegetable", "Fruit",
                 "Vegetable", "Fruit", "Fruit", "Vegetable", "Fruit"],
    "Qty": [12, 5, 10, 2, 10, 3, 5, 8, 2, 10],
    "Price": [350, 80, 320, 50, 200, 50, 280, 380, 60, 400]
})

# Two-way cross table: Category vs Product (count of occurrences)
print(pd.crosstab(df['Category'], df['Product']))
# Three-way cross table: Category vs Product, values=Qty, aggregation=sum
print(pd.crosstab([df['Category'], df['Product']], df['Qty']))
```

Perform various operation using date and time data types

```
import datetime as dt

now = dt.datetime.now()
print("Current datetime:", now)

today = dt.date.today()
print(" Today:", today, "| Day:", today.day,
      "| Month:", today.month, "| Year:", today.year)

# Difference between two dates
d1 = dt.date(2021, 5, 15)
d2 = dt.date(2020, 7, 25)
print("Date Difference:", d1 - d2)
```

Chapter 7 : Data Modelling

42. What is generative modeling?

- **Generative models** are a family of machine learning models used to describe how data is generated. It's an unsupervised learning method where the model automatically learns the rules and patterns from large datasets and can generate new data instances similar to the training data.

43. What is predictive modeling?

- **Predictive modeling** is a mathematical approach to building models from existing data to estimate or predict future outcomes or trends. It uses statistical techniques, including data mining and machine learning

44. List the types of predictive modeling.

1. Classification Model
2. Clustering Model
3. Forecast Model
4. Time Series Model
5. Outliers Model

45. What is histogram?

- **Histogram** is a graphical representation of the distribution of numerical data using rectangular bars where the height of each bar represents the frequency of data in a certain range (bin).

46. List types of charts.

- Histogram
- Scatter Plot
- Line Chart (Time Series Plot)
- Bar Chart

47. What is graph?

- **A graph** is a pictorial representation of a set of objects (vertices) connected by links (edges). It is used to model relationships between entities. In Python, graphs can be represented using dictionaries where vertices are keys and edges are values.

48. List the 3D plots.

- 3D Line Plot
- 3D Scatter Plot
- 3D Bar Plot
- Wire Plot
- Surface Plot

49. Explain predictive modeling in details.

- predictive modeling involves using historical data to build models that predict future outcomes. It includes:
 - **Classification Model:** Works with categorical data (fraud detection).
 - **Clustering Model:** Groups data based on shared attributes (customer segmentation).

- **Forecast Model:** Predicts numerical values (sales forecasting).
- **Time Series Model:** Analyzes data over time (seasonal trends).
- **Outliers Model:** Identifies anomalies (fraud detection).

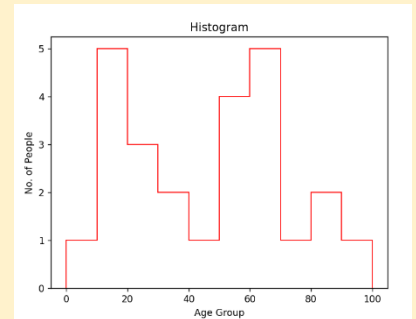
50. Explain histogram with example.

- **histogram** visualizes the distribution of numerical data by grouping values into bins

```
import matplotlib.pyplot as plt

age = [22, 55, 62, 45, 21, 22, 4, 12, 14, 64, 58, 68, 95, 85, 55, 38, 18, 3
7, 65, 59, 11, 15, 80, 75, 65]
bins = [0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100]

plt.hist(age, bins, color="red", histtype="step")
plt.xlabel("Age Group")
plt.ylabel("No. of People")
plt.title("Histogram")
plt.show()
```



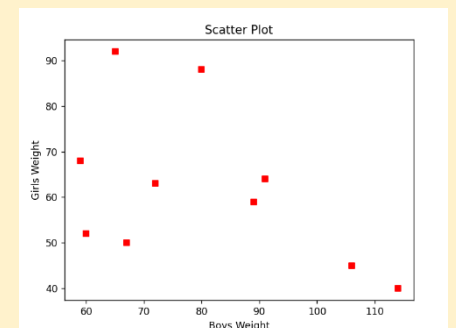
51. Explain scatter plot with example.

- **scatter plot** displays individual data points as dots to show relationships between two variables

```
import matplotlib.pyplot as plt

Boys_Weight = [67, 89, 72, 114, 65, 80, 91, 106, 60, 59]
Girls_Weight = [50, 59, 63, 40, 92, 88, 64, 45, 52, 68]

plt.scatter(Boys_Weight, Girls_Weight, color="red", marker="s")
plt.title("Scatter Plot")
plt.xlabel("Boys Weight")
plt.ylabel("Girls Weight")
plt.show()
```



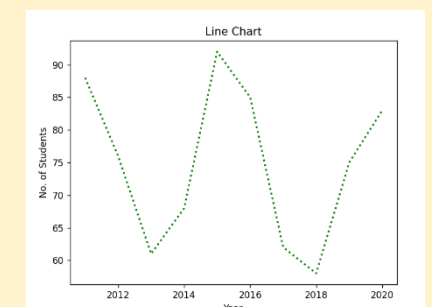
52. Explain line chart or time series plot with example.

- **line chart** shows trends over time

```
from matplotlib import pyplot as plt

Year = [2011, 2012, 2013, 2014, 2015, 2016, 2017, 2018, 2019, 2020]
Student = [88, 76, 61, 68, 92, 85, 62, 58, 75, 83]

plt.plot(Year, Student, linestyle="dotted", color="green", linewidth=2)
plt.title("Line Chart")
plt.xlabel("Year")
plt.ylabel("No. of Students")
plt.show()
```



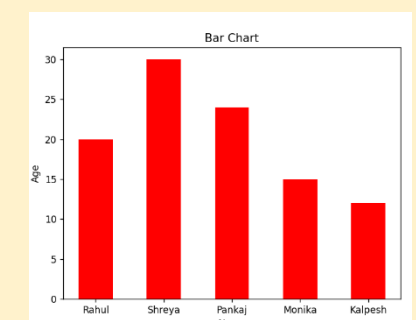
53. Explain bar plot with example.

- **bar plot** compares categories using rectangular bars

```
import matplotlib.pyplot as plt
import numpy as np

Name = np.array(["Rahul", "Shreya", "Pankaj", "Monika", "Kalpesh"])
Age = np.array([20, 30, 24, 15, 12])

plt.bar(Name, Age, color="red", width=0.5)
plt.title("Bar Chart")
plt.xlabel("Name")
plt.ylabel("Age")
plt.show()
```



54. Explain three-dimensional plot with example

- **3D plots** visualize data in three dimensions

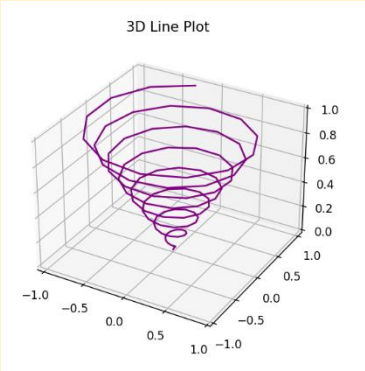
```
from mpl_toolkits import mplot3d
import numpy as np
import matplotlib.pyplot as plt

fig = plt.figure()
ax = plt.axes(projection="3d")

z = np.linspace(0, 1, 100)
x = z * np.sin(50 * z)
y = z * np.cos(50 * z)

ax.plot3D(x, y, z, "purple")
ax.set_title("3D Line Plot")

plt.show()
```



Question	Code
Create and display histogram with different arguments	<pre>import matplotlib.pyplot as plt age = [22, 55, 62, 45, 21, 22, 34, 42, 42, 4, 2, 102, 95, 85, 55, 110, 120, 7, 65, 55, 111, 115, 80, 75, 65, 4, 44, 43, 42, 48] bins = [0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100] plt.hist(age, bins, histtype="step", rwidth=0.8, color="red") plt.xlabel("Age Group") plt.ylabel("No. of People") plt.title("Histogram") plt.show()</pre>
Create and display scatter plot with different arguments	<pre>import matplotlib.pyplot as plt Boys_Weight = [67, 89, 72, 114, 65, 80, 91, 106, 60, 59] Girls_Weight = [50, 59, 63, 40, 92, 88, 64, 45, 52, 68] size = [200, 220, 240, 260, 280, 300, 320, 340, 360, 380] plt.scatter(Boys_Weight, Girls_Weight, s=size, alpha=0.5) plt.title("Scatter Plot") plt.xlabel("Boys Weight") plt.ylabel("Girls Weight") plt.show()</pre>
Create and display time series plot with different arguments	<pre>from matplotlib import pyplot as plt Year = [2011, 2012, 2013, 2014, 2015, 2016, 2017, 2018, 2019, 2020] Student = [88, 76, 61, 68, 92, 85, 62, 58, 75, 83] plt.plot(Year, Student, linestyle="dashed", color="green", linewidth=3) plt.title("Line Chart") plt.xlabel("Year") plt.ylabel("No. of Students") plt.show()</pre>
Create and display bar plot with different arguments	<pre>import matplotlib.pyplot as plt import numpy as np Name = np.array(["Rahul", "Shreya", "Pankaj", "Monika", "Kalpesh"]) Age = np.array([20, 30, 24, 15, 12]) plt.barh(Name, Age, color="skyblue", edgecolor="black", height=0.6) plt.title("bar plot") plt.xlabel("Age") plt.ylabel("Name") plt.show()</pre>
Create a simple graph	<pre>graph = { "a": ["b", "c"], "b": ["a", "c", "d"], "c": ["a", "b", "e"], "d": ["b", "e"], "e": ["c", "d"] } print(graph)</pre>
Create and display various 3D plots	<pre>import numpy as np import matplotlib.pyplot as plt fig = plt.figure() ax = plt.axes(projection="3d") x = np.linspace(-5, 5, 25) y = np.linspace(-5, 5, 25) X, Y = np.meshgrid(x, y) Z = np.sin(np.sqrt(X**2 + Y**2)) ax.plot_surface(X, Y, Z, cmap="viridis") ax.set_title("3D Surface Plot") plt.show()</pre>

Chapter 8 : Data Science Applications

الشبتر ده مفهوش أسئلة في الآخر / هو عبارة عن كلام نظرى على تطبيقات الداتا ساينس ف حاولت اعمل ملخص يغطى المحتوى (و يفضل نقراه برضه)

Sector	applications
Business	- Decision-Making: - Context-aware problem analysis. - Data quality measurement. - Use of algorithms (SAS, IBM, SPSS) for solutions. - Success stories to guide teams. - Product Development: - Customer feedback analysis. - Market trend evaluation. - Advanced analytics for innovation. - Business Management: - Performance tracking. - Growth prediction. - Global product distribution analysis. - Predictive Analytics: - Fraud detection. - Marketing campaign optimization. - Risk reduction. - Decision Assessment: - Evaluating prediction impacts. - Mitigating adverse consequences.
Clean Energy	- Optimization of solar/wind farm performance. - Predictive maintenance scheduling. - Cost and efficiency improvements via IoT and sensor data. - Environmental and economic benefits over fossil fuels.
Healthcare	- Medical Imaging: - Enhanced accuracy in X-rays, MRIs, and CT scans using deep learning. - Genomics: - DNA sequence analysis (tools: MapReduce, SQL, Galaxy). - Bioinformatics for drug-gene interaction studies. - Drug Discovery: - Accelerated research (e.g., COVID-19 vaccines). - Genetic mutation profiling for targeted drugs. - Patient Monitoring: - IoT devices track vital signs (heartbeat, temperature). - Disease Prevention: - Early detection of chronic diseases via AI.
Insurance	- Fraud Detection: Historical data analysis to identify fraudulent claims. - Price Optimization: Algorithmic adjustment of premiums based on customer data. - Risk Assessment: Classification of risks (pure/speculative) for policy setting. - Claims Prediction: Models (e.g., decision trees, SVM) forecast financial losses. - Healthcare Insurance: Analyzes claims data and medical records for cost reduction.
Travel	- Viewpoint Analysis: Sentiment analysis of customer reviews. - Tour Planning: AI-driven cost-effective itineraries based on user history
Transport	- Self-driving cars. - Fuel consumption and driver behavior analysis. - Route optimization to avoid traffic.
Manufacturing	- Price Optimization: Balances material, labor, and production costs. - Predictive Analytics: Forecasts market trends and demand.

	• Inventory Management: Prevents overproduction and optimizes supply chains.
Telecommunication	• Fraud Detection: ML identifies unauthorized access. • Predictive Analytics: Historical data analysis for decision-making. • Customer Segmentation: Groups users by value, behavior, or lifecycle.
Supply Chain Management	• Predicting delays. • Managing inventory. • Storing backup data for emergencies.
Gaming	• Game Development: Player behavior analysis and algorithm design. • Monetization: Predictive analytics for user engagement. • Fraud Detection: Secures payment systems via ML. • Graphics: Real-time rendering and motion capture.
Governance	• Secure data management. • Organizational workflow efficiency. • Stakeholder trust via transparent data usage.
Biotechnology	• Red: Medical • White: Industrial • Green: Agricultural • Gold: Bioinformatics • Blue: Marine/aquatic applications. • Yellow: Nutritional biotechnology.
Pharmaceuticals	• Patient selection via social media/data. • Real-time monitoring. • Early side-effect prediction.
Geospatial Analytics	• Telecom: Network issue visualization. • Military: Resource placement optimization. • Urban Planning: Population impact analysis. • Weather: Disaster response and airline routing.